

SaaS conversion plan for RMCOrder

RMCOrder can become a strong vertical SaaS product for small apparel brands, print shops, and merchandise businesses. The existing application already contains valuable domain logic: order construction, decoration placement, blank inventory, design management, production emails, customer status notifications, and demand-based purchasing.

The major change is architectural:

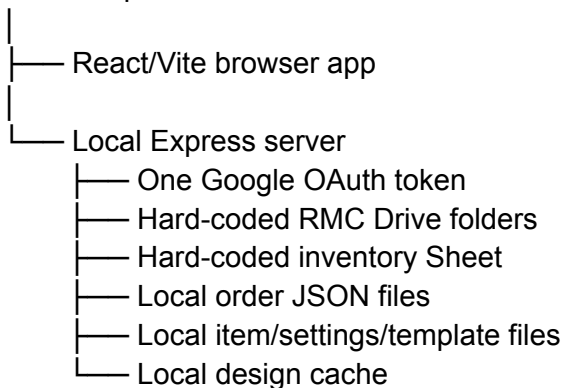
Google Drive, Sheets, local JSON files, and one shared Google account must stop being the application's core database and identity system.

The SaaS version should use a multi-tenant PostgreSQL database as its source of truth, object storage for artwork, a dedicated account/organization system, Stripe for subscriptions, and background workers for email and integrations. Google Drive, Gmail, Sheets, and Square can remain available as optional per-store integrations.

1. What the application is today

The current product is a trusted, single-company desktop/server application:

One RMC operator



The domain functionality is reusable, but the infrastructure assumes:

- One business.
- One Google account.
- One item catalog.
- One inventory.
- One set of settings.

- One email identity.
- One printer recipient.
- One globally numbered order sequence.
- One machine with persistent local storage.
- Trusted users and trusted network access.

Those assumptions appear throughout:

- [server/config.js](C:/PERSONAL_INTEREST/Programming/RMCOOrder/server/config.js) contains global Google folders, a global inventory Sheet, local token paths, and local data paths.
- [server/auth/oauth.js](C:/PERSONAL_INTEREST/Programming/RMCOOrder/server/auth/oauth.js) stores one Google token set in `tokens.json`.
- [server/middleware/requireAuth.js](C:/PERSONAL_INTEREST/Programming/RMCOOrder/server/middleware/requireAuth.js) checks only whether that global refresh token exists.
- [server/orders/cache.js](C:/PERSONAL_INTEREST/Programming/RMCOOrder/server/orders/cache.js) stores orders as local JSON.
- [server/items/store.js](C:/PERSONAL_INTEREST/Programming/RMCOOrder/server/items/store.js) and [server/settings/store.js](C:/PERSONAL_INTEREST/Programming/RMCOOrder/server/settings/store.js) are global filesystem stores.
- [server/sheets/router.js](C:/PERSONAL_INTEREST/Programming/RMCOOrder/server/sheets/router.js) trusts submitted order data and externally supplied Sheet IDs.
- [server/inventory/router.js](C:/PERSONAL_INTEREST/Programming/RMCOOrder/server/inventory/router.js) updates one global Sheet without database transactions or concurrency protection.
- [src/hooks/useOfflineQueue.js](C:/PERSONAL_INTEREST/Programming/RMCOOrder/src/hooks/useOfflineQueue.js) keeps pending changes only in memory, so refreshing the page loses the queue.
- [server/settings/router.js](C:/PERSONAL_INTEREST/Programming/RMCOOrder/server/settings/router.js) currently has no authentication middleware and exposes a server-side `git pull/npm install` update operation. That endpoint must not exist in a hosted SaaS service.

These are not failures in the current local product. They are simply the boundary between a personal application and a public multi-tenant system.

2. Recommended production stack

My recommended “practical small-team SaaS” stack is:

Concern	Recommended service	Why
Frontend	Vercel or Cloudflare Pages	Excellent React/Vite deployment, CDN, preview environments
API	Render web service	Minimal rewrite for the existing Express backend
Worker	Render background worker	Email, Google/Square synchronization, exports, retries
Database	Supabase managed PostgreSQL	Relational data, migrations, backups, optional storage/realtime
ORM/schema	Drizzle ORM + PostgreSQL migrations	Type-safe, relatively lightweight, SQL remains understandable
Authentication	Clerk	Accounts, sessions, organizations, invitations, membership roles
Billing	Stripe Billing	Subscriptions, trials, invoices, taxes, coupons, customer portal
File storage	Supabase Storage initially	Easy private buckets and signed URLs; move to R2/S3 later if needed
Queue	BullMQ + managed Redis	Reliable async work, retries, rate limiting, scheduled jobs
Redis	Upstash Redis or Render Key Value	Queue state, locks, rate limits, short-lived caching
Product email	Postmark or Resend	Reliable transactional email and delivery reporting
User Gmail connection	Google OAuth	Optional “send/draft through my Gmail” integration
Retail/POS	Square OAuth/API	Per-store catalog, inventory and sales integration
Error tracking	Sentry	Frontend/backend errors, traces, releases
Product analytics	PostHog	Funnels, adoption, feature flags, optional session replay
Logs	Render logs initially; Axiom later	Structured searchable production logs

DNS/WAF	Cloudflare	DNS, bot filtering, rate limiting, domain protection
CI/CD	GitHub Actions	Tests, linting, migrations, deployment gates
Secrets	Hosting-provider secrets initially	Later move to Doppler/1Password Secrets if needed

Clerk’s organization model is designed around multi-tenant applications with members, invitations, roles, and permissions, which closely matches “a user creates a store and invites employees.” [Clerk organization documentation](#)

Supabase’s PostgreSQL support also gives you row-level security as a second isolation layer. Supabase explicitly recommends enabling RLS on exposed tables; policies effectively add tenant conditions to each query. [Supabase RLS documentation](#)

Stripe should be the authority for payment events, but your database should hold a local projection of subscription status. Stripe subscriptions are asynchronous and must be synchronized through signed, idempotently processed webhooks. [Stripe subscriptions](#), [Stripe subscription webhooks](#)

Render supports both ordinary Node/Express services and continuously running background workers, letting you preserve most of this backend while separating user requests from slow Google, Square, and email operations. [Render Express deployment](#), [Render background workers](#)

Why I would not rewrite everything into serverless functions

The current backend has:

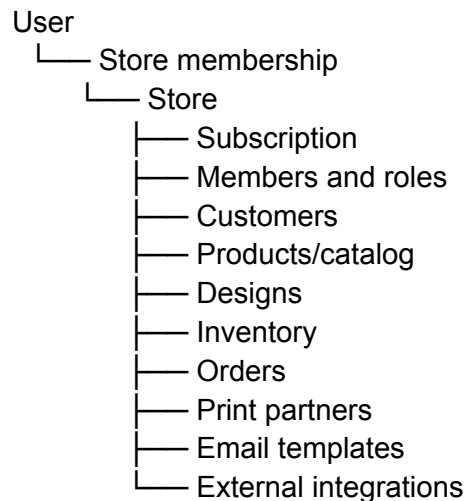
- Google API operations.
- Bulk file copying.
- Sheet synchronization.
- Email construction.
- Artwork synchronization.
- Calculations over uploaded CSVs.
- Operations that may take longer than a typical request.
- A need for background retry behavior.

A normal Express API plus a worker is a better fit than converting every endpoint into independent serverless functions. You can still host the frontend separately on a CDN.

3. Target product model

The fundamental tenant should be a **store**, not an individual user.

A person may belong to multiple stores. A store may have several employees. Billing, data, integrations, settings, files, order numbering, and usage limits belong to the store.



Suggested roles:

- **owner**: billing, delete store, integrations, members, all business data.
- **admin**: all operational data and most settings; no ownership transfer.
- **manager**: orders, inventory, designs, customers, templates.
- **staff**: create and edit orders; limited settings access.
- **viewer**: read-only reporting and order visibility.

Do not encode permissions only in React. Every backend mutation must verify:

1. The session is valid.
2. The user belongs to the requested store.
3. The membership has the required permission.
4. The resource itself belongs to that store.
5. The store subscription permits the operation.

4. Recommended database design

Use UUID or time-sortable UUID primary keys internally. Keep human-readable order numbers as separate scoped identifiers.

Every tenant-owned table must contain `store_id`, directly or through an unambiguous parent. I prefer direct `store_id` on important operational tables because it simplifies isolation, indexing, auditing, and deletion.

Identity and tenancy

users

- `id`
- `auth_provider_user_id`
- `email`
- `name`
- `avatar_url`
- `created_at`
- `last_seen_at`

stores

- `id`
- `name`
- `slug`
- `timezone`
- `currency`
- `country`
- `order_prefix`
- `next_order_number`
- `status`
- `created_by`
- `timestamps`

store_memberships

- `store_id`
- `user_id`
- `role`
- `status`
- `timestamps`

Unique constraint on (`store_id`, `user_id`).

subscriptions

- `store_id`
- `stripe_customer_id`
- `stripe_subscription_id`
- `stripe_price_id`
- `plan`
- `status`
- `trial_ends_at`
- `current_period_ends_at`
- `cancel_at_period_end`
- `grace_period_ends_at`
- timestamps

Never trust plan or subscription status supplied by the browser.

Store configuration

store_settings

- Branding and logo file ID.
- Store reply-to address.
- Printer defaults.
- Default decoration notes.
- Email behavior.
- Inventory policies.
- Locale/timezone.
- Feature configuration.

print_partners

Replace the RMC-specific `spewEmail` setting with:

- Name.
- Email recipients.
- Contact data.
- Default decoration instructions.
- Delivery method.
- Optional Drive sharing rules.
- Active/inactive state.

An order should reference a print partner, not a hard-coded Spew configuration.

Catalog and artwork

catalog_items

- `store_id`
- Name.
- SKU/style.
- Supplier.
- Supplier URL.
- `stock_blanks`.
- Decoration methods.
- Active state.
- Metadata JSON.

catalog_item_colors

- `catalog_item_id`
- Name.
- Normalized name.
- Hex.
- Supplier code.
- Active.

catalog_item_sizes

- `catalog_item_id`
- Size.
- Sort order.
- Active.

designs

- `store_id`
- Name.
- Original filename.
- Storage object key.
- MIME type.
- Size.
- Checksum.
- Width/height.

- Thumbnail key.
- Tags.
- Archived state.
- Created by.

Files remain private. The API returns short-lived signed URLs.

Customers

customers

- `store_id`
- Name.
- Email.
- Phone.
- Marketing/notification preferences.
- External Square/customer ID.
- Notes.
- timestamps.

Avoid using email as the relational identifier. Emails change and can be shared.

Orders

orders

- `store_id`
- Internal ID.
- Human order number.
- Name.
- State.
- Delayed-from state.
- Print partner ID.
- Notes.
- Created by.
- Version.
- Sent/fulfilled/received/shipped timestamps.
- Soft-deletion timestamps.
- Legacy Google folder/sheet IDs during migration.

Unique (`store_id`, `order_number`).

Use a transaction or database sequence/locking mechanism when allocating the next store order number. Do not scan existing names as the current Drive implementation does.

order_line_items

- `order_id`
- `store_id`
- Position.
- Catalog item reference.
- Snapshotted item name/style/color.
- Customer ID.
- Front/back method and notes.
- State if line-level production status is retained.
- timestamps.

Snapshot display fields so historical orders do not change when catalog names change.

order_line_item_sizes

- `line_item_id`
- Size.
- Total quantity.
- Inventory quantity.
- Quantity-to-order, either generated or calculated.
- Unit cost/price later.

Constraints:

- `total_quantity >= 0`
- `inventory_quantity >= 0`
- `inventory_quantity <= total_quantity`

order_line_item_designs

- `line_item_id`
- `design_id`
- Placement: `front` or `back`.
- Sequence/design number.
- Snapshot filename.

order_customers

Use this only if an order can involve customers with no assigned line items. Otherwise relationships through line items may be sufficient.

order_events

Append-only event history:

- State changes.
- Email drafted/sent.
- Inventory allocation.
- User edits.
- Export to Sheets.
- Printer submission.
- Integration failures.

This becomes both the user-visible activity feed and part of the audit trail.

Inventory

inventory_locations

Stores may later have multiple warehouses, retail locations, or studios.

inventory_variants

- `store_id`
- Catalog item.
- Color.
- Size.
- Location.
- Quantity on hand.
- Quantity reserved.
- Reorder point.

inventory_transactions

Use a ledger rather than just updating a count:

- `receive`
- `allocate`
- `release`
- `consume`

- `adjust`
- `return`

Each transaction references an order when applicable. Update the balance and write the transaction atomically.

This fixes race conditions in the current Sheet read-modify-write implementation.

Messaging and jobs

`email_templates`

Store-specific, versioned templates by event type.

`messages`

- Store.
- Order/customer.
- Recipient.
- Provider.
- Provider message ID.
- Status.
- Template/version.
- Rendered subject/body or immutable content reference.
- Sent/error timestamps.

`jobs`

Optional application-level job tracking even if Redis runs the actual queue.

`integration_connections`

- `store_id`
- Provider.
- External account ID.
- Status.
- Scopes.
- Encrypted token data or vault reference.
- Token expiry.
- Last synchronized time.
- Last error.

webhook_events

- Provider.
- Provider event ID.
- Received time.
- Processing state.
- Payload hash/body.
- Attempts.

Unique provider event ID makes webhook processing idempotent.

audit_logs

- Store.
- Actor.
- Action.
- Resource type/ID.
- Safe change summary.
- IP.
- User agent.
- Request ID.
- Timestamp.

Do not place OAuth tokens, card data, passwords, or full sensitive payloads in audit logs.

5. Authentication and onboarding

Separate application login from external integrations.

Currently, Google OAuth performs both jobs at once:

- It acts like login.
- It gives access to Google Drive, Sheets, and Gmail.

In the SaaS version:

Application identity

Use Clerk for:

- Email/password or passwordless login.

- Google social login with only identity scopes.
- Email verification.
- Password reset.
- MFA.
- Session management.
- Organizations/stores.
- Invitations.
- Role membership.

Integration identity

Provide separate buttons inside store settings:

- Connect Google Drive.
- Connect Gmail.
- Connect Square.

These connection tokens belong to the store, not the logged-in user's application session.

Onboarding flow

1. User creates an account.
2. User verifies email.
3. User creates a store.
4. Store slug and timezone are selected.
5. Trial subscription is created.
6. Default roles/settings/templates are seeded.
7. User imports or creates catalog items.
8. User uploads a logo and designs.
9. User optionally connects Square, Google Drive, and Gmail.
10. User creates their first print partner.
11. User creates a first order.
12. Contextual checklist shows remaining setup.

Do not require Google connectivity just to use the product.

6. Billing and subscription handling

Use Stripe Checkout for signup and the Stripe Customer Portal for self-service management. Do not build card-entry or invoice-management UI yourself.

Suggested plans:

Starter

- One store.
- Two users.
- Limited active orders or monthly orders.
- Basic design storage.
- Transactional customer email.
- CSV import/export.

Growth

- More users.
- Square integration.
- Google Drive/Sheets export.
- Inventory and demand planning.
- Automated customer email.
- Multiple print partners.
- Expanded storage.

Pro

- Multiple locations.
- Advanced permissions.
- Higher limits.
- Custom branding.
- API/webhooks.
- Priority support.
- Advanced analytics.

Initially, price by store and included seats rather than taking a percentage of order value. The app manages production orders but does not currently process customer checkout revenue, so a transaction fee would be harder to justify.

Billing lifecycle

Handle at minimum:

- `checkout.session.completed`
- `customer.subscription.created`
- `customer.subscription.updated`
- `customer.subscription.deleted`

- `invoice.paid`
- `invoice.payment_failed`

Rules:

- Verify Stripe webhook signatures.
- Store every event ID before processing.
- Ignore duplicates.
- Update the local subscription projection transactionally.
- Allow a short grace period after payment failure.
- Restrict expensive mutations after cancellation but preserve read/export access temporarily.
- Never immediately delete store data because payment failed.
- Run a nightly reconciliation job against Stripe in case a webhook was missed.

Plan enforcement belongs on the backend. Frontend hiding is only convenience.

7. Google, email, and Square integrations

Google Drive and Sheets

Make Google optional.

Recommended model:

- PostgreSQL is always the source of truth.
- Files are always stored in your object storage.
- “Export/sync to Google Drive” is an integration feature.
- A store chooses a Drive folder during setup.
- Store Google file IDs in integration mapping tables.
- Run export/sync in jobs, not normal page requests.
- Show sync state and errors in the UI.

Avoid bidirectional Sheets editing in the first SaaS release. It creates conflict resolution, deletion, schema-change, and permission complexity. Start with one-way export or snapshot generation.

If bidirectional synchronization becomes necessary later:

- Store an external revision/version.
- Record `last_synced_at`.

- Detect conflicts.
- Never silently overwrite both sides using last-write-wins.
- Provide a conflict resolution UI.

Request the narrowest Google scopes possible. The existing full Drive scope is broad. Prefer application-created file access when product requirements permit it.

Email

Support two modes:

Hosted transactional email

Use Postmark or Resend for:

- Customer status emails.
- Invitations.
- Password/account notifications where not handled by Clerk.
- Billing and operational notices.

This gives consistent delivery, bounce handling, suppression lists, and provider webhooks.

Connected Gmail

Keep Gmail as an optional premium integration when a store wants:

- Drafts to appear in its Gmail account.
- Messages sent directly from its Google identity.
- Printer-order drafts owned by the operator.

All send operations should become queued jobs with idempotency keys. The UI should show **queued**, **sending**, **sent**, **failed**, and **drafted**.

Add unsubscribe or notification-preference handling where legally applicable. Transactional production updates and marketing emails must not be treated identically.

Square

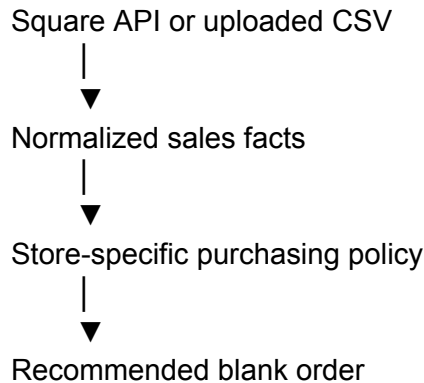
Implement proper Square OAuth per store.

Store:

- Merchant/account ID.
- Access token encrypted at rest.

- Refresh information if applicable.
- Granted scopes.
- Connected locations.
- Cursor or last synchronization point.
- Webhook signature configuration.

The existing CSV-based facts-versus-policy calculation is a good abstraction. Preserve it:



Do not make the blank-order algorithm dependent directly on Square response objects. Create a provider-neutral demand interface and retain CSV upload as a fallback.

8. API redesign

Move from resource identifiers such as Sheet IDs to application-owned IDs:

```

GET    /api/v1/stores/:storeId/orders
POST   /api/v1/stores/:storeId/orders
GET    /api/v1/stores/:storeId/orders/:orderId
PATCH /api/v1/stores/:storeId/orders/:orderId
DELETE /api/v1/stores/:storeId/orders/:orderId
POST   /api/v1/stores/:storeId/orders/:orderId/transitions
POST   /api/v1/stores/:storeId/orders/:orderId/emails
  
```

The backend must derive or verify **storeId** through membership. Merely including a tenant ID in the URL is not authorization.

Additional API rules:

- Validate all request bodies with Zod.
- Reject unknown or malformed fields.
- Return stable error codes, not raw exception messages.

- Use pagination for orders, customers, designs, logs, and messages.
- Add request IDs.
- Add optimistic concurrency through a `version` column or `updated_at` precondition.
- Use idempotency keys for create-order, send-email, inventory mutations, and billing-related actions.
- Generate signed upload URLs for direct artwork upload.
- Limit upload size and accepted MIME types.
- Normalize filenames; never treat filenames as trusted paths.
- Apply store and user rate limits.

The current whole-order snapshot can remain useful at first, but gradually normalize writes. An interim `orders.document JSONB` field could reduce migration risk; do not leave all operational querying permanently trapped in JSON.

9. Offline behavior

The current in-memory queue is not durable. For SaaS:

- Store pending mutations in IndexedDB.
- Give every mutation a UUID idempotency key.
- Include the order version on updates.
- Persist the queue across reloads.
- Process mutations in order.
- Use exponential backoff.
- Display the last confirmed server save time.
- Distinguish “browser offline” from “server rejected the change.”
- Do not repeatedly retry validation, permission, or subscription errors.
- Surface conflicts if the server order changed since the offline edit.

For the first public release, it may be safer to support ordinary retry and optimistic concurrency before promising full offline editing. Offline multi-user conflict handling can become a later feature.

10. Security work required before public beta

Immediate fixes

Before even staging the public application:

- Remove `/settings/update` from production.
- Require authentication on every settings route.
- Replace the global token-file authentication model.
- Remove hard-coded tenant Drive and Sheet identifiers.
- Do not serve the local designs directory publicly.
- Validate all order, catalog, inventory, blank-order, settings, and email payloads.
- Stop returning raw third-party error messages to clients.
- Add rate limiting and request size limits by route.
- Restrict CORS to configured production origins.
- Add secure response headers.
- Ensure production errors do not expose stacks, tokens, paths, or Google payloads.

Session and browser protections

- Secure, HttpOnly, SameSite cookies if using cookie sessions.
- CSRF protection for cookie-authenticated mutations.
- Strict Content Security Policy.
- `frame-ancestors 'none'` unless embedding is intentional.
- HSTS.
- MIME sniffing protection.
- Referrer policy.
- Dependency scanning.
- No OAuth tokens in browser storage.
- No service-role database key in frontend code.

Tenant isolation

Use two layers:

1. Every API repository query includes `store_id`.
2. PostgreSQL RLS prevents cross-store access if an API query is wrong.

Automated isolation tests should create Store A and Store B, then attempt every read and mutation across store boundaries.

Secrets and integration tokens

- Encrypt Google and Square refresh/access tokens using envelope encryption.

- Store the encryption master key outside the database.
- Support key versioning and rotation.
- Redact authorization headers and tokens from logs.
- Revoke tokens when an integration disconnects.
- Verify OAuth `state` and use PKCE where supported.
- Allow only registered redirect URIs.
- Provide users a visible integration activity and disconnect control.

Files

- Private buckets only.
- Short-lived signed URLs.
- Image MIME and signature verification.
- File size/dimension limits.
- Randomized object keys, not raw filenames.
- Malware scanning for arbitrary uploads.
- Thumbnail processing in a worker.
- Orphaned-upload cleanup job.
- Per-plan storage quotas.

Email and webhooks

- Verify Stripe and Square webhook signatures using the raw request body.
- Process asynchronously.
- Enforce idempotency.
- Protect against replay.
- Rate-limit outgoing emails per store.
- Track bounce and complaint events.
- Do not let a user submit arbitrary sender identities without verification.

Business and compliance basics

Before commercial launch, prepare:

- Terms of Service.
- Privacy Policy.
- Data Processing Addendum.
- Subprocessor list.
- Data deletion and export process.
- Retention policy.
- Security contact.
- Incident response plan.

- Backup restoration procedure.
- Basic accessibility review.
- Colorado/US privacy-law review with counsel.
- CAN-SPAM and applicable email requirements.
- Stripe Tax or another tax strategy where SaaS tax applies.

Do not pursue SOC 2 before product-market fit unless a target customer requires it. Build SOC 2-friendly controls—access logging, least privilege, change control, backups—without paying for formal certification immediately.

11. Reliability and operations

Background jobs

Move these out of HTTP requests:

- Email sending and Gmail draft creation.
- Drive export and synchronization.
- Square synchronization.
- Design thumbnail processing.
- Spreadsheet generation.
- Blank demand recomputation.
- Subscription reconciliation.
- Data export.
- Account deletion.
- Backup verification.

Each job should have:

- Store ID.
- Actor ID if user-triggered.
- Job type.
- Idempotency key.
- Attempt count.
- Backoff.
- Timeout.
- Structured error.
- Dead-letter handling.

Observability

Every request and job should carry:

- Request/job ID.
- Store ID.
- User ID where applicable.
- Route/job name.
- Duration.
- Status.
- Safe error code.

Set alerts for:

- Elevated API error rate.
- Failed Stripe webhooks.
- Queue backlog.
- Repeated email failures.
- Database connection exhaustion.
- High latency.
- Storage quota anomalies.
- Failed backups or restore drills.
- Google/Square token refresh failures.

Use Sentry for exceptions and tracing. Use PostHog for product behavior, but avoid capturing customer email bodies, artwork, order notes, or other sensitive fields in analytics.

Backups

- Managed daily database backups.
- Point-in-time recovery when revenue justifies it.
- Object storage versioning or retention.
- Quarterly restore test initially, then monthly.
- Documented recovery-time and recovery-point targets.
- Separate backup of configuration needed to reconstruct infrastructure.
- Export capability per store.

A backup is not verified until you have successfully restored it.

12. CI/CD and environments

Create three isolated environments:

- Local development.
- Staging.
- Production.

Each needs separate:

- Database.
- object-storage buckets.
- Clerk application.
- Stripe keys/products/webhook endpoints.
- Google OAuth client.
- Square application.
- email provider credentials.
- Redis/queue.
- domains.

GitHub Actions should run:

1. Dependency installation from lockfiles.
2. Frontend tests.
3. Backend tests.
4. Lint.
5. Type checking once TypeScript is introduced.
6. Production build.
7. Database migration validation.
8. Security/dependency scan.
9. Deploy staging.
10. Smoke tests.
11. Controlled production deployment.

Do not run migrations blindly from multiple application instances. Use a single release/migration job.

13. Recommended implementation phases

Phase 0 — Product definition and threat model

Estimated: 1–2 weeks.

Decide:

- Exact target customer: apparel brand, print shop, or both.
- Whether each store represents a brand or a production shop.
- Whether printers log in or only receive emails.
- Which features belong in the first paid plan.
- Data ownership and retention.
- Whether Google export is mandatory.
- Pricing metric.
- Required roles.
- Initial support model.

Deliverables:

- Product requirements.
- Tenant/data ownership map.
- Threat model.
- Architecture decision records.
- Initial pricing and entitlement matrix.

Phase 1 — Make the current backend deployable

Estimated: 2–3 weeks.

- Parameterize ports, domains, and OAuth redirects.
- Remove the updater endpoint.
- Add authenticated settings access.
- Add Zod validation.
- Add centralized error handling.
- Add structured logging and request IDs.
- Add security headers and rate limiting.
- Dockerize the API.
- Serve frontend and backend through production domains.
- Configure staging CI/CD.
- Add health, readiness, and build-version endpoints.

This phase does not yet make the app multi-tenant. It creates a safe deployment base.

Phase 2 — PostgreSQL persistence

Estimated: 4–6 weeks.

- Establish Supabase PostgreSQL.
- Introduce Drizzle migrations.
- Implement stores, users, memberships, settings, catalogs, customers, designs, orders, line items, and inventory.

- Create repository/service layers.
- Preserve existing calculation and email-builder functions as domain modules.
- Dual-write or import current JSON/Sheets data.
- Compare imported database records against existing order snapshots.
- Switch reads to PostgreSQL.
- Keep Google Sheets as export/fallback during verification.
- Implement database backups and restore test.

Priority: move orders and tenant configuration first; move reporting later.

Phase 3 — Accounts and stores

Estimated: 3–4 weeks.

- Add Clerk.
- Create store onboarding.
- Add store switcher.
- Add invitations.
- Add role/permission middleware.
- Introduce tenant-scoped URLs and queries.
- Add RLS and isolation tests.
- Replace global Google authentication with application sessions.
- Add account/store deletion and export foundations.

At the end of this phase, separate test companies must be unable to access each other's data.

Phase 4 — Object storage and design migration

Estimated: 2–3 weeks.

- Create private buckets.
- Add signed uploads/downloads.
- Store metadata and checksums.
- Generate thumbnails.
- Migrate cached and Drive designs.
- Update design browser.
- Add quotas and deletion lifecycle.
- Add optional one-way Drive export.

Phase 5 — Billing

Estimated: 2–3 weeks.

- Create Stripe products and prices.
- Add Checkout.
- Add customer portal.
- Implement webhook ingestion and reconciliation.
- Add trials and grace periods.
- Add entitlement middleware.
- Add plan/usage UI.
- Test cancellation, upgrade, downgrade, failed payments, and duplicates.

Use Stripe test clocks or equivalent testing features to exercise subscription transitions.

Phase 6 — Jobs, email, and integrations

Estimated: 4–6 weeks.

- Add Redis and worker.
- Queue customer email and printer drafts.
- Add hosted transactional email.
- Add optional Gmail OAuth.
- Add Square OAuth and webhook processing.
- Preserve CSV upload.
- Add integration status, reconnect, and error UI.
- Add retry/dead-letter handling.

Phase 7 — Inventory correctness and collaboration

Estimated: 3–5 weeks.

- Replace Sheet inventory with transactional ledger.
- Implement atomic allocation/release.
- Add optimistic concurrency to orders.
- Persist offline operations in IndexedDB.
- Add activity history.
- Add multi-user conflict feedback.
- Add location support only if early customers need it.

Phase 8 — Private beta hardening

Estimated: 3–4 weeks.

- Recruit 3–10 pilot stores.
- Add Sentry and operational alerts.
- Run tenant isolation security tests.

- Perform a lightweight penetration test.
- Run backup restore and disaster simulation.
- Add admin/support tooling with audited impersonation or secure support views.
- Add data export/deletion.
- Complete legal documents.
- Measure onboarding completion and first-order success.
- Establish support response expectations.

Phase 9 — Public launch

- Public marketing and pricing pages.
- Production status page.
- Help center and onboarding videos.
- In-app support/contact.
- Usage metering.
- Churn and failed-payment flows.
- Error-budget and incident process.
- Regular dependency and access reviews.

A realistic small-team timeline is roughly four to seven months for a credible paid beta, depending on whether one person is working part-time or full-time. A narrower MVP can launch sooner by postponing bidirectional Google sync, advanced offline support, multi-location inventory, and printer portals.

14. Migration strategy for existing RMC data

Treat RMC as Tenant 1.

1. Create the RMC store.
2. Import store settings and email templates.
3. Import the item catalog.
4. Import designs and compute checksums.
5. Import customers.
6. Import each cached order snapshot.
7. Preserve original order IDs.
8. Store legacy Drive folder and Sheet IDs.
9. Convert sizes and designs into normalized child records.
10. Import current inventory.

11. Recalculate aggregate totals and compare them with source data.
12. Run the local and SaaS systems in read-only comparison mode.
13. Switch RMC to database-backed reads.
14. Keep one-way Google exports enabled temporarily.
15. Archive—not immediately delete—the original Drive/Sheet data.

Write a repeatable import program with a dry-run report. Do not manually edit production tables.

Validation should compare:

- Order count by state.
 - Line item count.
 - Size totals.
 - Inventory pulls.
 - Customer assignments.
 - Design references.
 - Email history where available.
 - Current inventory totals.
 - Blank-order calculations.
-

15. What should be reused versus rewritten

Reuse with limited modification

- React components and overall workflows.
- Order state normalization and transition concepts.
- Line-item compile logic.
- Blank-order demand and policy algorithms.
- Email rendering functions.
- Color parsing and item utilities.
- Existing frontend/backend test fixtures.
- Customer-to-line-item association concepts.
- Current order preview behavior.

Wrap behind new interfaces

- Order persistence.
- Catalog persistence.

- Settings/templates persistence.
- Inventory operations.
- Design storage.
- Gmail client.
- Google Drive client.
- Square demand source.
- Email delivery.

For example:

```
class OrderRepository {
  list(storeId, filters) {}
  create(storeId, actorId, input) {}
  get(storeId, orderId) {}
  update(storeId, orderId, expectedVersion, input) {}
}
```

Business logic should depend on this interface, not on Google Sheets or filesystem paths.

Retire from production

- Local JSON as authoritative storage.
- Global `tokens.json`.
- Global `settings.json`.
- Global `items-catalog.json`.
- Public local design cache.
- Hard-coded Google IDs.
- Runtime `git pull/npm install`.
- Scanning Drive folder names to generate the next order number.
- Fire-and-forget promises inside API processes.
- Arbitrary whole-order replacement without schema and ownership validation.

16. Testing strategy

Keep the existing 266 test declarations as a valuable regression base, then add:

- Database repository tests.
- Migration/import fixture tests.
- Tenant isolation tests.
- Permission matrix tests.

- Stripe webhook tests.
- Duplicate/reordered webhook tests.
- Google/Square token refresh tests.
- Inventory concurrency tests.
- Order version-conflict tests.
- Worker retry/dead-letter tests.
- Signed file URL tests.
- Browser-level onboarding and first-order tests.
- Backup restore smoke test.
- End-to-end tests using Playwright.

The highest-value security test is:

For every tenant-owned endpoint, prove that a valid Store A user cannot read, mutate, email, export, or delete a Store B resource even when they know its ID.

17. Cost expectations

Exact vendor pricing will change, so verify it at implementation time. Architecturally, expect:

Development/private alpha

Approximately \$50–\$150/month:

- Small managed PostgreSQL.
- One API service.
- One small Redis instance.
- Low-volume email.
- Basic monitoring.
- Static frontend hosting.

Early paid beta

Approximately \$150–\$500/month:

- Production database and backups.
- API and background worker.
- Redis.
- Auth usage.
- Email volume.

- Object storage.
- Monitoring/logging.
- Staging environment.

Growth

Costs will primarily scale with:

- Active users.
- Database size and compute.
- Artwork storage and bandwidth.
- Email volume.
- Worker workload.
- Log retention.
- Square/Google synchronization frequency.

Keep a gross-margin dashboard tracking infrastructure cost per active store. Storage and email quotas should exist before they become expensive, even if initial plan limits are generous.

18. Recommended MVP boundary

For the first paid version, I would ship:

- Account creation and verified login.
- One store per subscription.
- Owner/admin/staff roles.
- Store-scoped catalog, customers, designs, inventory, and orders.
- Private artwork storage.
- Transactional customer status emails.
- Printer order email.
- Stripe subscription and customer portal.
- CSV-based Square sales import.
- Optional one-way Google Drive/Sheets export.
- Audit history.
- Backups, monitoring, exports, and deletion process.

I would postpone:

- Bidirectional Google Sheets synchronization.
- Fully offline collaborative editing.
- Multi-location inventory.

- Custom domains.
- Customer storefront/e-commerce checkout.
- Printer-facing portal.
- Public API.
- Supplier automation.
- Mobile native applications.
- Advanced revenue analytics.
- Enterprise SSO and SOC 2 certification.

That boundary preserves the existing product's strongest features while keeping the first SaaS version achievable.

19. The first ten implementation tickets

If development started tomorrow, I would create these first:

1. Remove the production update endpoint and protect all settings routes.
2. Add centralized validation, errors, request IDs, logging, headers, and rate limiting.
3. Create the PostgreSQL schema and migration tooling.
4. Add `stores`, `users`, `memberships`, and store-scoped settings.
5. Build database-backed catalog and order repositories.
6. Write an RMC JSON/Sheet import and validation tool.
7. Add Clerk sessions, store creation, invitations, and permission middleware.
8. Add automated tenant-isolation tests across every route.
9. Replace local design serving with private object storage and signed URLs.
10. Deploy a staging frontend, API, database, worker, and monitoring stack.

The critical path is:

Security baseline

- PostgreSQL
- Store tenancy
- Account authorization
- Object storage
- Billing
- Background jobs/integrations
- Private beta

The most important architectural decision is to make PostgreSQL the source of truth and turn Google/Square/Gmail into tenant-specific adapters. Once that boundary exists, most of the existing RMC business logic can evolve into a real SaaS product without requiring a complete rewrite.

